

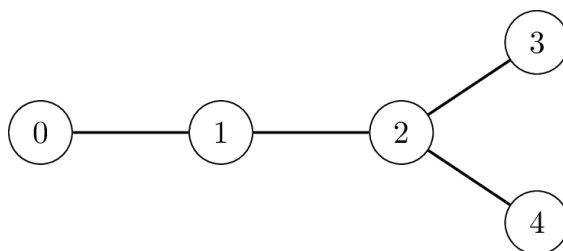
Міграції

Музей природознавства досліджує шляхи міграції динозаврів у Болівії. Палеонтологи виявили сліди динозаврів на N різних місцях, пронумерованих від 0 до $N - 1$ у порядку зменшення віку: місце 0 містить найстаріші сліди, а місце $N - 1$ - наймолодші.

Динозаври мігрували на кожне місце (крім місця 0) з певного, більш старішого місця. Для кожного місця i ($1 \leq i \leq N - 1$), існує рівно одне старіше місце $P[i]$ (де $P[i] < i$), так що деякі динозаври мігрували безпосередньо з місця $P[i]$ на місце i . Одне старіше місце могло бути джерелом міграції на кілька молодших місць.

Палеонтологи моделюють кожну міграцію як **неорієнтований зв'язок** між місцями i та $P[i]$. Зауважте, що для будь-яких двох різних місць x та y можна подорожувати з x до y через послідовність з'єднань. **Відстань** між місцями x та y визначається як мінімальна кількість з'єднань, необхідних для подорожі з x до y .

Наприклад, на наступному зображенні показано випадок з $N = 5$ місць, де $P[1] = 0$, $P[2] = 1$, $P[3] = 2$ і $P[4] = 2$. Від місця 3 до місця 4 можна потрапити через 2 з'єднання, тому відстань між ними дорівнює 2.



Музей прагне визначити пару місць з максимально можливою відстанню. Зверніть увагу, що пара не обов'язково має бути унікальною: наведеному вище прикладі обидві пари $(0, 3)$ та $(0, 4)$ мають відстань 3, яка є максимальною. У таких випадках будь-яка з пар, що досягла максимальної відстані, вважається правильною.

Спочатку значення $P[i]$ **невідомі**. Музей відправляє дослідницьку групу відвідати місця $1, 2, \dots, N - 1$, у такому порядку. Досягнувши кожного місця i ($1 \leq i \leq N - 1$), команда виконує обидві наступні дії:

- Визначає значення $P[i]$, тобто джерело міграції на місце i .

- Вирішує, чи надсилати **одне** повідомлення до Музею або не надсилати жодних повідомлень з цього місця на основі попередньо зібраної інформації.

Повідомлення передаються через дорогу систему супутникового зв'язку, тому кожне повідомлення має бути цілим числом від 1 до 20 000 включно. Крім того, команді дозволяється надіслати **не більше 50 повідомлень** загалом.

Ваше завдання - реалізувати стратегію, за допомогою якої:

- Дослідницька група вибирає місця, з яких надсилаються повідомлення, а також значення кожного повідомлення.
- Музей може визначити пару місць з максимальною відстанню, ґрунтуючись виключно на повідомленнях, отриманих з кожного місця, знаючи, з яких місць були надіслані ці повідомлення.

Надсилання великої кількості повідомлень через супутник коштує дорого. Ваш результат буде залежати як від найбільшого надісланого цілого числа, так і від загальної кількості переданих повідомлень.

Деталі реалізації

Вам потрібно реалізувати дві функції: одну для дослідницької групи, а іншу - для музею.

Функція, яку ви повинні реалізувати для **дослідницької групи**, є такою:

```
int send_message(int N, int i, int Pi)
```

- N : кількість місць зі слідами.
- i : індекс місця, на якому зараз перебуває команда.
- Pi : значення $P[i]$.
- Ця функція викликається $N - 1$ разів для кожного тесту в порядку $i = 1, 2, \dots, N - 1$.

Ця функція повинна повернути $S[i]$ із зазначенням дій, виконаних дослідницькою групою на місці i :

- $S[i] = 0$: дослідницька група вирішує не відправляти повідомлення з місця i .
- $1 \leq S[i] \leq 20\,000$: дослідницька група відправляє ціле число $S[i]$ як повідомлення з місця i .

Функція, яку ви повинні реалізувати для **музею**, є такою:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : масив довжиною N такий, що:
 - $S[0] = 0$.

- Для кожного $1 \leq i \leq N - 1$, $S[i]$ є цілим числом, яке повертає `send_message(N, i, P[i])`.
- Ця функція викликається рівно один раз для кожного тесту.

Ця функція повинна повернути пару місць (U, V) з максимальною відстанню.

Під час фактичного оцінювання програма, яка викликає наведені вище функції, виконується **рівно два рази**.

- Під час першого запуску програми
 - `send_message` викликається рівно $N - 1$ разів.
 - **Ваша програма може зберігати інформацію під час наступних викликів**
 - Значення, що повертаються (масив S), зберігаються системою оцінювання.
 - У деяких випадках поведінка градера є **адаптивною**. Це означає, що значення $P[i]$ при виклику `send_message` можуть залежати від дій, виконаних дослідницькою групою під час попередніх викликів.
- Під час другого запуску програми:
 - `longest_path` викликається рівно один раз. Єдиною інформацією, доступною для `longest_path` є масив S з першого запуску.

Обмеження

- $N = 10\,000$
- $0 \leq P[i] < i$ для всіх i , таких що $1 \leq i \leq N - 1$.

Підзадачі та оцінювання

Підзадача	Бали	Додаткові обмеження
1	30	Місце 0 та деяке інше місце мають максимальну відстань серед усіх пар місць.
2	70	Без додаткових обмежень.

Нехай Z - максимальне ціле число, що міститься у масиві S , а M - кількість повідомлень, надісланих дослідницькою групою.

У будь-якому з тестових випадків, якщо виконується хоча б одна з наведених нижче умов, оцінка вашого розв'язку для цього тестового випадку буде 0 (у CMS відображається як `Output isn't correct`):

- Принаймні один елемент у S є невірним.
- $Z > 20\,000$ або $M > 50$.
- Значення, що повертається при виклику функції `longest_path` є некоректним.

В іншому випадку ваш бал за підзадачу 1 розраховується наступним чином:

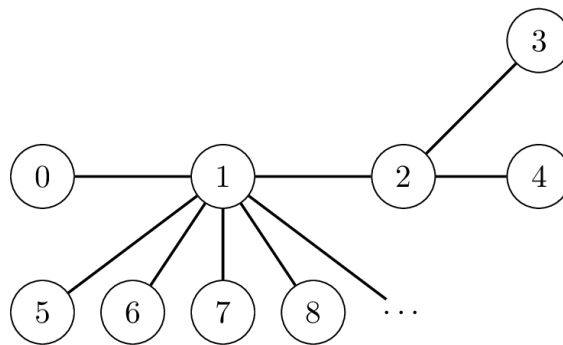
Обмеження	Бали
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Ваш бал за виконання підзадачі 2 розраховується наступним чином:

Обмеження	Бали
$5 \leq Z \leq 20\,000$ та $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ та $32 \leq M \leq 50$	40
$Z \leq 4$ та $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ та $M \leq 8$	70

Приклад

Нехай $N = 10\,000$. Розглянемо ситуацію, у якій $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, і $P[i] = 1$ для $i > 4$.



Припустимо, що стратегія дослідницької групи полягає в тому, щоб надсилати повідомлення $10 \cdot V + U$ щоразу, коли пара місць (U, V) з максимальною відстанню змінюється в результаті виклику `send_message`.

Спочатку парою з максимальною відстанню є $(U, V) = (0, 0)$. Розглянемо наступну послідовність викликів під час першого запуску програми:

Виклик	(U, V)	Повернуте значення ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Зверніть увагу, що у всіх інших викликах значення $P[i]$ дорівнює 1. Це означає, що пара з максимальною відстанню не змінюється, і команда більше не надсилає жодних повідомлень.

Потім, під час другого запуску програми, виконується наступний виклик:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Музей читає останнє повідомлення, надіслане дослідницькою групою, а саме $S[3] = 30$, і робить висновок, що $(0, 3)$ - це пара місць з максимальною відстанню. Таким чином, виклик повертає $(0, 3)$.

Зауважте, що такий підхід не завжди дозволяє музею правильно визначити пару з максимальною відстанню.

Приклад градера

Приклад градера викликає і `send_message`, і `longest_path` в одному запуску, що відрізняється від реального градера.

Формат вхідний даних:

```
N
P[1] P[2] ... P[N-1]
```

Формат вихідних даних:

```
S[1] S[2] ... S[N-1]
U V
```

Зверніть увагу, що ви можете використовувати приклад градера з будь-яким значенням N .